

Fonctionnement du système

Stumps aléatoires + règle à 4 cas, sans routeur

1 Le trajet d'un exemple, du dataset à la sortie

On utilise partout dans ce document le même petit dataset jouet : $n = 4$ features booléennes $x = (x_0, x_1, x_2, x_3)$, une étiquette $y \in \{0, 1\}$, et une clause déjà créée avec $K = 2$ features conditionnées, tirées au hasard : $\{0, 2\}$, fixées respectivement à $(1, 1)$.

clause (exemple fil rouge)	
features conditionnées ($K = 2$)	$\{0, 2\}$, fixées à $(1, 1)$
features libres ($n - K = 2$)	$\{1, 3\}$
condition <code>applies(x)</code>	vraie ssi $x_0=1$ et $x_2=1$

Contrairement à l'ancienne architecture à routeur, **aucun calcul de gain, aucune structure partagée** ne détermine ce choix : $\{0, 2\}$ et leurs valeurs sont un tirage `uniform_int_distribution` pur, indépendant de toute autre clause déjà créée.

Voici tout le trajet d'un exemple à travers cette clause *déjà entraînée*. Une clause, c'est : une condition figée sur K features (section 2) qui décide si la clause s'applique du tout à cet exemple, et un simple **ET logique** de littéraux appris sur les $n - K$ features libres (section 3).

Exemple fil rouge 1.1. On prend $x = (1, 1, 1, 0)$. La condition de la clause fil rouge teste $x_0=1$ (vrai) et $x_2=1$ (vrai) : la clause **s'applique**. Supposons qu'elle ait appris, sur ses deux features libres $\{1, 3\}$: $v[1] = \text{Include}$ (le littéral " x_1 ") et $\bar{v}[3] = \text{Include}$ (le littéral " $\neg x_3$ "), tout le reste exclu.

1.1 Que veut dire "littéral inclus" ?

Chaque feature libre i possède deux automates : $v[i]$ (associé au littéral x_i) et $\bar{v}[i]$ (associé au littéral $\neg x_i$). Chacun a un état entier dans $\{1, \dots, 2N\}$: au-dessus de N , le littéral est Include (il compte dans la clause) ; à N ou en-dessous, il est Exclude (il est ignoré). La clause est le ET logique de tous les littéraux inclus. Elle est **violée** dès qu'un seul littéral inclus est faux pour x ; elle est **satisfaite** seulement si *tous* ses littéraux inclus sont vrais pour x .

2 Le conditionnement : un stump aléatoire, pas un routeur

C'est le choix architectural central de tout le système. Une approche par arbre de décision (routeur), même bien conçue, présente une pathologie prouvée empiriquement : quand K (la profondeur/le nombre de coupures) devient assez grand, le découpage *seul* devient assez puissant pour résoudre le problème — les clauses deviennent décoratives. La solution retenue : **éliminer toute adaptation** dans le choix des conditions.

2.1 Le tirage, à la création de chaque clause

2.2 La porte d'entrée : `applies(x)`

$$\text{applies}(x) = \bigwedge_{k=1}^K [x_{f_k} = v_k]$$

où (f_k, v_k) sont les K paires (feature, valeur requise) tirées à la création. Un exemple qui ne satisfait pas `applies(x)` est **invisible** pour cette clause : ni pour l'entraînement (le sous-ensemble applicable l'exclut), ni pour le vote final (section 6).

Note 2.1. Le nombre de combinaisons possibles de K features parmi n est $\binom{n}{K}$. La probabilité qu'un tirage tombe sur une combinaison réellement informative décroît donc avec $\binom{n}{K}$ — c'est la "loterie combinatoire" qui impose M (nombre de clauses) grand quand $K > 1$ ou n grand (ex. Mux11 : $\binom{11}{3} = 165$, $M = 8000$ nécessaire pour une couverture correcte des 8 combinaisons d'adresse).

3 Mettre à jour une clause : la règle à 4 cas, masquée

En dehors des features conditionnées, une clause apprend exactement la règle à 4 cas démontrée dans *convergence_sans_bruit.tex* — `updateMasked` se contente de sauter les features où `active[i]=false`.

$x[i]$	y	Action sur $v[i]$ (littéral x_i)	Action sur $\bar{v}[i]$ (littéral $\neg x_i$)
0	0	<code>towardInclude(S)</code>	<code>towardExclude(S)</code>
0	1	<code>towardExclude(α)</code>	rien
1	0	<code>towardExclude(S)</code>	<code>towardInclude(S)</code>
1	1	rien	<code>towardExclude(α)</code>

Exemple fil rouge 3.1. Reprenons $x = (1, 1, 1, 0)$, applicable à la clause fil rouge, avec $y = 1$ (vrai label). Seules les features libres $\{1, 3\}$ sont mises à jour :

- $i = 1$: $x_1=1, y=1 \Rightarrow$ rien sur $v[1]$, et $\bar{v}[1]$ fait `towardExclude( $\alpha$ )` ;
- $i = 3$: $x_3=0, y=1 \Rightarrow v[3]$ fait `towardExclude( $\alpha$ )`, rien sur $\bar{v}[3]$.

$v[0], \bar{v}[0], v[2], \bar{v}[2]$ (conditionnées) ne bougent jamais, quel que soit x ou y .

3.1 Propriété clé : l'indépendance totale entre automates

Le mouvement d'un automate ne dépend **que** de (x_i, y) — jamais de la sortie de la clause $C(x)$, jamais de l'état des autres automates $j \neq i$, jamais des autres clauses. C'est la différence structurelle avec la vraie Tsetlin Machine (feedback Type I/II, où l'automate reçoit un feedback qui dépend de la sortie de *toute* sa clause, section 3.3 du papier original) : chez nous, chaque paire $(v_i, \bar{v}_i)$ exécute sa **propre chaîne de Markov indépendante**, directement réductible à une marche aléatoire biaisée à barrières réfléchissantes (voir *convergence_sans_bruit.tex*, Lemme 3.1). C'est cette propriété qui rend :

- l'entraînement **trivialement parallélisable** (une clause = un thread, zéro synchronisation) ;
- la preuve de convergence **directement généralisable** d'un seul bit à n features libres : chaque automate libre converge indépendamment, la convergence jointe de la clause entière suit par intersection (cas sans bruit) ou union bound (cas bruité).

3.2 Convention : clause vide = abstention

Une clause où **aucun** littéral n'est inclus (ni sur les features conditionnées — elles ne le sont jamais par construction — ni sur les libres) est traitée comme **vide** : elle s'abstient du vote (section 6), au lieu de retourner 1 par défaut comme le ferait un calcul naïf de $\text{output}(\mathbf{x})$.

4 Entraîner : tirage équilibré, total mises à jour

Le tirage à l'intérieur du sous-ensemble applicable suit le même principe que dans l'ancienne architecture (*anatomie_moteur.tex*, §6) : 50/50 entre les deux classes, puis pondéré par les poids w courants à l'intérieur de chaque classe.

5 Mesurer l'erreur, pondérer (AdaBoost par clause)

Différence majeure avec l'ancienne architecture : l'erreur et la repondération ne portent **que** sur le sous-ensemble applicable de la clause — les poids des exemples hors de ce sous-ensemble restent *inchangés* pour ce round.

$$\text{err} = \frac{\sum_{i \in \text{applicable, mal classé}} w_i}{\sum_{i \in \text{applicable}} w_i} \in [10^{-6}, 0.999], \quad \alpha = \frac{1}{2} \ln \left(\frac{1 - \text{err}}{\text{err}} \right).$$

$$w_i \leftarrow \begin{cases} e^\alpha & \text{si } i \text{ applicable, mal classé} \\ e^{-\alpha} & \text{si } i \text{ applicable, bien classé} \\ 1 & \text{si } i \text{ non applicable (inchangé)} \end{cases} \quad \text{puis renormalise } \sum_i w_i = 1.$$

Note 5.1. Chaque clause est donc son **propre round de boosting complet**, et non un composant d'un round partagé. Une clause chanceuse (tombée sur une bonne combinaison de features) obtient un α élevé ; une clause malchanceuse (condition sur des features non pertinentes) obtient $\alpha \approx 0$ et pèse presque rien dans le vote final — sans jamais perturber les poids des exemples hors de son sous-ensemble.

6 Combiner les M clauses et décider

$$\text{score}(x) = \sum_{\substack{m=1 \\ \text{applies}_m(x) \wedge \neg \text{isEmpty}_m}}^M \alpha_m \cdot (2 \cdot C_m(x) - 1), \quad \hat{y} = \begin{cases} 1 & \text{si } \text{score}(x) > 0 \\ 0 & \text{sinon.} \end{cases}$$

6.1 Extension multi-classe (one-vs-rest)

Pour Iris, Digits et MNIST : un ensemble *indépendant* de M clauses par classe c , entraîné avec l'étiquette relabellisée $y' = \mathbb{I}[y = c]$.

$$\hat{y} = \operatorname{argmax}_{c \in \{1, \dots, \text{numClasses}\}} \text{score}_c(x).$$

Chaque classe a son propre ensemble de clauses, ses propres tirages de conditionnement, ses propres α — aucun partage entre classes.

7 Hyperparamètres et meilleures configurations trouvées

Paramètre	Contrôle	Ce qu'on a appris
M	nombre de clauses	compense la loterie combinatoire ($\binom{n}{K}$)
K	features conditionnées / clause	$K=1$ suffit si n petit ; K plus grand débloque des
$S \in [0, 1]$	proba. de mouvement vers Include	probabilité réelle — $S=1$ quasi toujours optimal
$\alpha \in [0, 1]$	proba. de mouvement vers Exclude	optimum dataset-dépendant, trouvé par balayage
N	demi-plage d'états	trop grand + total fixe $\Rightarrow$ jamais saturé à temps
total	tirages d'entraînement / clause	doit rester assez petit pour ne pas saturer aveuglément

Dataset	M	K	S	α	N	total	Accuracy (20 runs)
XOR	200	1	1.0	0.35	50	1500	99.07% $\pm$ 2.16%
Iris	1500	2	1.0	0.30	100	20000	95.5% $\pm$ 4.25%
Mux11	8000	3	1.0	0.30	100	20000	99.97% $\pm$ 0.02%
Parity3	1000	2	1.0	0.30	100	20000	91.9% $\pm$ 7.4%
Digits	300	1	1.0	0.30	100	20000	92.3–92.6% ($K=2$ en cours)
MNIST	40000	1	1.0	0.30	256	20000	90.3% (budget complet du papier)

Portée : `random_stump_perclause_boost.cpp` · `random_stump_multifeature.cpp` · `random_stump_multiclass.cpp` · `random_stump_multifeature_multiclass.cpp` — tous réutilisant `Clause` et `updateMasked` de `tsetlin_engine.h`. Appliqué identiquement aux six datasets. Contrairement à *anatomie_moteur.tex* (architecture précédente), **aucun routeur, à aucun niveau.**

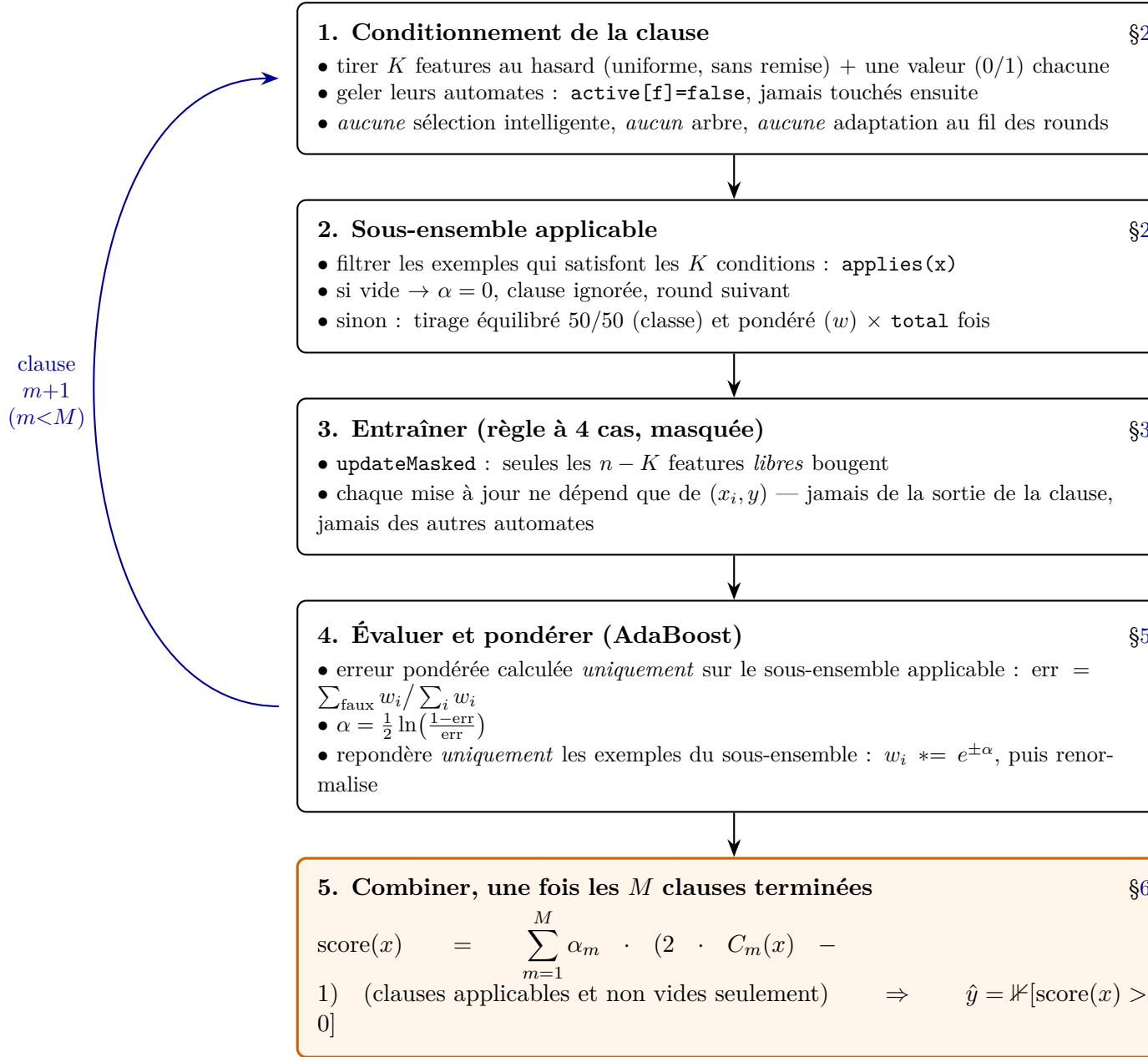


Figure 1: Vue d’ensemble complète : chaque clause $m = 1, \dots, M$ est un round AdaBoost **totalelement indépendant** des autres — aucune étape ne nécessite de connaître l’état d’une autre clause. C’est la différence structurelle centrale avec la vraie Tsetlin Machine (feedback Type I/II, couplage via la somme globale des clauses) et avec notre ancienne architecture à routeur (voir *anatomie_moteur.tex*) : ici, **aucun routeur**, à **aucun niveau**.

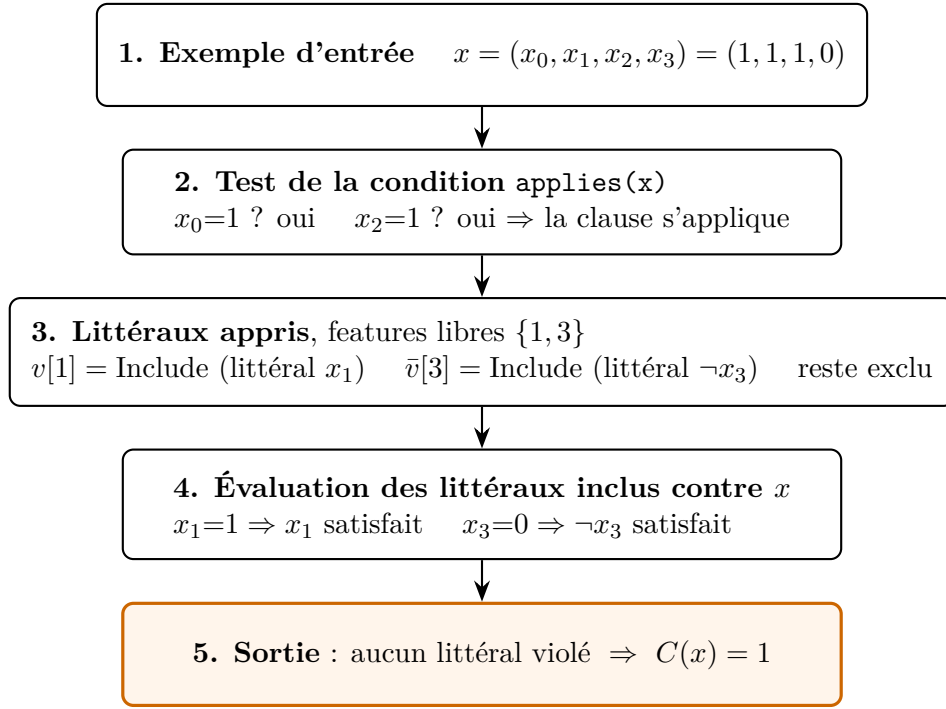


Figure 2: Le trajet complet d'un exemple, du dataset à la sortie d'une clause. Si $\text{applies}(x)$ avait été fausse à l'étape 2, la clause se serait **abstenue** : ni entraînement, ni vote, pour cet exemple précis.

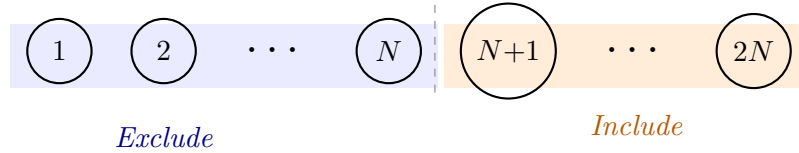


Figure 3: L'état d'un automate se lit sur une seule droite : au-dessus de N , Include; à N ou en-dessous, Exclude. Identique pour les features conditionnées (figées à l'état 1, donc toujours Exclude, absentes de la clause) et pour les features libres (mobiles).

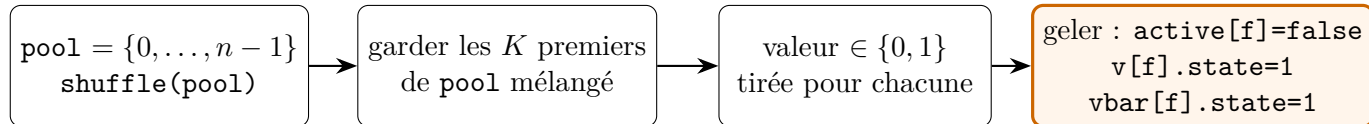


Figure 4: Aucun calcul de gain, aucune information sur y n'intervient dans ce tirage — contrairement au routeur de l'ancienne architecture (Gini, gain $\times$ masse, *anatomie_moteur.tex* § 3), le choix est purement **uniform_int_distribution**.

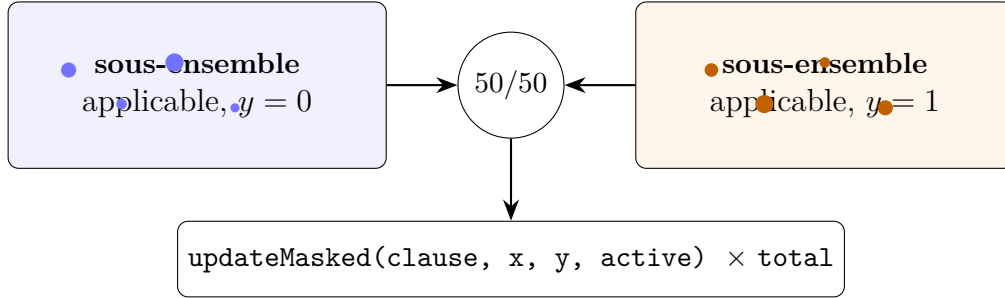


Figure 5: Contrairement à l’ancienne architecture, le tirage se fait *exclusivement* à l’intérieur du sous-ensemble applicable de **cette** clause — jamais sur tout le dataset.

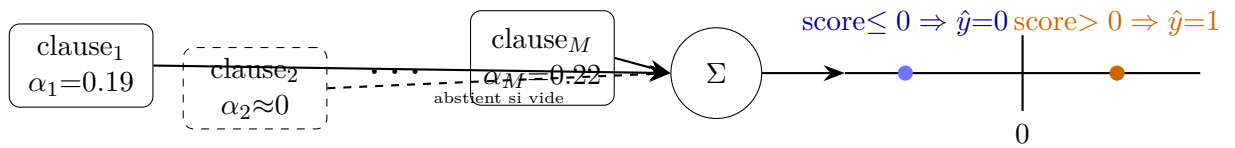


Figure 6: Vote pondéré final. Les clauses non applicables à x , ou vides, **n’entrent pas dans la somme** (représentées en pointillé) — elles ne votent ni pour 0 ni pour 1, elles s’abstiennent purement et simplement.